SPRING 2026

SCALABLE AI SERVICES WITH AGENTIC AI, MLOPS & LLMOPS

CS5305/CS621 - PROGRAMMING ASSIGNMENT 2

CS5305/CS621
SPRING 2026
LUMS SBASSE

1. Lab Overview

This lab will focus on creating a fault-tolerant, continuous data pipeline utilizing Apache Spark Structured Streaming and Delta Lake. Students will implement a Medallion Architecture that ingests raw data, processes CDC events, and aggregate temporal data for reporting with a continuous stream of data.

Key Learning Objectives

- Incremental Ingestion: Utilizing Databricks Auto Loader to track and ingest asynchronous file arrivals.
- State Transition & CDC: Resolving out-of-order logs and in-batch duplicate events using MERGE INTO and Spark Window Functions
- Temporal Aggregation: Calculating sliding window metrics governed by strict event-time watermarks

Key Technologies

Databricks Autoloader

Spark Structured Streaming is the **general-purpose streaming engine in Apache Spark** that processes incremental data from many kinds of sources (Kafka, Delta, files, sockets, etc.) using the DataFrame API

Auto Loader is a **Databricks-optimized file ingestion source built *on top of Structured Streaming***, specifically designed for **scalable, reliable cloud object storage ingestion** (ADLS, S3, GCS).

While the Autoloader API mentions format as always “CloudFiles”, the location is what determines where it will read the data from, and the location can be any external cloud storage, or unity catalog volume.

Ref: https://learn.microsoft.com/en-us/azure/databricks/ingestion/cloud-object-storage/auto-loader/

You have will option to use either spark structured streaming or autoloader to build your solution.

Pre-requisites Setup: Workspace & Storage Initialization

Task: Setup Unity Catalog hierarchy and file system required for the streaming data pipeline.

Task: Create your solution notebook and create the following objects:

- **Catalog & Schemas:** Initialize a new catalog with the naming convention: `<rollnumber>-PA2`.
- **Schemas:** Within this catalog, create three schemas: `bronze`, `silver`, and `gold`
- **Volumes:** Initialize a volume named `temp` within your catalog.
- **Directories:** Within the `temp` volume, create the following four subdirectories:
    - `flight_landing_zone:` Stores flight (.csv) data files
    - `flight_landing_zone_part2:` Stores flight (.csv) data files for part 2.
    - `cdc_zone_part2:` Stores flight CDC (.csv) data files for part 2.
    - `flight_reference_data:` Stores static referential (.txt) data files.
    - `checkpoints:` Stores checkpoints for each write stream.
    - `schemas:` Stores the schemas of data.

PART 1: Stream Analytics

Task 1: Scalable Ingestion with Auto Loader (Bronze Layer)

Objective: Establish a robust streaming read from the landing zone to capture raw data files as they arrive.

Requirements:

- Load stream data (departuredelays.csv) from "/databricks-datasets/flights" into the `flight_landing_zone` directory.
- Configure Databricks Auto Loader (`cloudFiles`) to monitor the `flight_landing_zone` directory for `.csv` files.
- Define the `cloudFiles.schemaLocation` and set the `cloudFiles.schemaEvolutionMode` to `rescue` to handle unexpected schema changes gracefully.
- Append an `ingestion_timestamp` column to the stream to track exact data arrival times. Format this timestamp strictly as `yyyy-MM-dd HH:mm:ss`
- **Write** the raw streaming DataFrame to a Delta table named `flights` in the `bronze` schema using `append` mode.
- Configure the checkpoint location in the appropriate directory and set the stream trigger mode to `availableNow=True`

Questions:

1. What is schemaEvolutionMode used for, and under what circumstances should it be used?
2. What kinds of trigger modes are there, and what is the purpose of each?

Task 2: Data Cleansing, Enrichment, and Standardization (Silver Layer)

Part 1: Type Casting and Temporal Standardization

Objective: Clean the raw Bronze data, enforce strict data types, and establish the event timeline.

Requirements:

- Cast the numeric and string columns (delay, distance, origin, destination, etc.) to their appropriate data types.
- The raw `date` column lacks a year. Attach the year "2025" to the string, parse it, and convert it into a standard timestamp format: `yyyy-MM-dd HH:mm:ss`.
- Maintain the `ingestion_timestamp` column. Create a new `event_timestamp` column based on the standardized flight date.

Part 2: Referential Enrichment and Routing Logic

Objective: Utilize static reference data to map IATA codes to geographic locations and standardize flight paths.

Requirements:

- Using `/airport-codes-na.txt` from "/databricks-datasets/flights", retrieve City and State for both origin and destination IATA codes for the streaming silver data.
- Create two new columns, `departure` and `arrival`, formatted exactly as: `City, State`. Filter out any empty departure or arrival values
- Create a `take_off_status` column that categorizes the flight based on the `delay` time using the following boundaries:
    - **Early:** Departed before scheduled time (`delay < 0`).
    - **On Time:** Departed on time or slightly delayed (`0 <= delay < 10` minutes).
    - **Late:** Noticeably delayed (`10 <= delay < 30` minutes).
    - **Delay:** Severely delayed (`delay >= 30` minutes).

- Create a `standardized_route` column. Because the direction of the flight does not matter for downstream aggregations, sort the origin and destination IATA codes alphabetically and concatenate them with a hyphen (e.g., "JNU-SEA").
- Initialize a sequence column named `seq` and set its default literal value to 3 for CDC preparation.
- Select the final columns in this exact order: `date, departure, arrival, standardized_route, distance, delay, take_off_status, ingestion_timestamp, event_timestamp, seq`.
- **Write** the transformed stream to a Delta table named `flights` in the `silver` schema. Configure the appropriate checkpoint location and set the stream to trigger to availableNow=True.

Question:

1. How does a broadcast join optimize performance?

Task 3: Stateful Streaming Aggregations (Gold Layer)

Objective: Transform the version-controlled Silver data into refined, business-level analytical reports (the Gold Layer). You will utilize the event_timestamp to perform event-time windowing and apply appropriate watermarks to mathematically handle late-arriving data.

General Requirements:

- All streaming queries must read from your established Silver Delta table stream.
- You must enable checkpointing. Each query **must** write to a unique, isolated checkpoint directory

Requirement 1: Average Delay per Month

Summary: Create a streaming aggregation that calculates the moving average of flight delays (Hours) across the entire network, grouped by a one-month tumbling window.

- Create a delta table named `monthly_delays`
- You must convert the recorded delay from minutes into hours.
- The final table must exactly display the following columns:
  - `window_start` (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - window_end (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - `moving_avg_delay` (Rounded to 2 decimal places)

Requirement 2: Total Route Distance Every 2 Weeks

Summary: Aggregate the cumulative flight distance covered across all routes within discrete 14-day intervals.

- Create a Delta table named biweekly_distance.
- Group the incoming stream using a 2-week temporal window and calculate the sum of the distance column.
- The final table must exactly display the following columns:
  - window_start (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - window_end (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - total_distance (rounded to 0 decimal places)

Requirement 3: Monthly Maximum Delay by Destination

Summary: Identify the absolute maximum delay incurred within each monthly window and isolate the specific route responsible for that latency.

- Create a Delta table named monthly_max_delay.
- Convert the delay into **hours**. You will need to utilize an Argmax operation (e.g., the max_by expression) to extract the string of the worst-performing route without breaking the temporal grouping.

- The final table must exactly display the following columns:
  - `window_start` (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - `window_end` (Formatted as a String: yyyy-MM-dd HH:mm:ss)
  - `worst_route` (The standardized route string)
  - `max_delay` (The maximum delay in hours, rounded to 2 decimal places)

Task 4: Task: Tumbling Window Aggregation for Hourly Flight Counts (Gold Layer)

Objective: Use event-time–based tumbling windows to compute the number of flights per hour.

PART 3: Change Data Capture

Task 1: Continuous Stream Orchestration

Objective: Implement a decoupled, real-time data pipeline. You will utilize a `data_ingestion` notebook that continuously generates flight data and Change Data Capture (CDC) updates, and create a `etl_stream_processor` notebook that continuously processes this data through your Medallion architecture.

Notebook A: Data Ingestion

Objective: Simulate an active data ingestion framework.

Requirements:

- Create a new notebook named `data_ingestion`.
- Paste the provided `generate_flight_data` and `generate_cdc_updates` functions exactly as provided (Do not modify them).
- Configure the paths to save the flight data and CDC updates to their appropriate directories (`flight_landing_zone_part2`, `cdc_zone_part2`)

Notebook B: The Stream Processor

Objective: Execute your Medallion architecture continuously, reacting to the Producer's data feed.

Requirements:

- Create a second notebook named `etl_stream_processor`.
- Assemble your Bronze, Silver, and Gold streaming logic into this notebook.
- Schedule the notebook to run on regular intervals
- Create a function named `merge_cdc_to_silver` that reads a batch of CDC stream data using the `.forEachbatch` Function and writes the changes to the silver table. The function should:
  1. Parse the json payload object into an appropriate schema.
  2. Deduplicate any recurring data and keep the data with the highest sequence number.
  3. Based on the action, update or delete an existing record.
  4. Apply the CDC data with the highest sequence number only to handle out-of-order CDC logs.